

Confidence Limits

Today's agenda:

- Confidence limits (7.5)

Last time

We discussed issues with model fitting and how to deal with them.

Today, we'll work on understanding confidence limits of a model

(Previously we've talked about PARAMETER confidence limits)

Let's start with some data and a model

```
library(emdbook)
library(bbmle)
data(GobySurvival)

# subset data to a single density treatment, single experiment, and better than median quality

dat <- subset(GobySurvival,
              exper == 1 &
              density == 9 &
              qual > median(qual))

# estimate survival time based on the last time it was observed and the last time it wasn't
observed

time <- (dat$d1 + dat$d2)/2
```

Let's start with some data and a model

For our model we'll use a Weibull distribution

- Similar to Gamma
- Models time to failure

```
weiblikfun <- function(shape, scale) {  
  -sum(dweibull(time,  
               shape = shape,  
               scale = scale,  
               log = TRUE)  
)  
}
```

Fit our model using mle2

```
w1 <- mle2(weiblikfun,  
           start = list(shape = 1,  
                         scale = mean(time)))
```

Run the your code and make sure your model fit alright

Getting parameters and confidence intervals

Does anyone remember how to get the model parameters?

How about the confidence intervals on the model parameters?

Getting parameters and confidence intervals

Model parameter estimates: `summary(w1)`

Model parameter CIs: `confint(w1)`

What are the 95% confidence intervals for the 2 parameters?

Getting parameters and confidence intervals

	estimate	2.5%	97.5%
shape	0.92074	0.6248949	1.281101
scale	14.37754	8.0469975	24.766768

From parameter uncertainty to function uncertainty

- We now know the uncertainty of the shape and scale parameters
- But how do we get the uncertainty of survival?
- Options:
 - Re-parameterize your model (see 7.5.1.1): involves math, works for some models
 - Constrain/penalize the model (see 7.5.1.2): a coding hassle
 - Derivative-based estimates (see 7.5.2): requires derivative stuff
 - Population Prediction intervals: work for anything

From parameter uncertainty to function uncertainty

- We now know the uncertainty of the shape and scale parameters
- But how do we get the uncertainty of survival?
- Options:
 - Re-parameterize your model (see 7.5.1.1): involves math, works for some models
 - Constrain/penalize the model (see 7.5.1.2): a coding hassle
 - Derivative-based estimates (see 7.5.2): requires derivative stuff
 - **Population Prediction intervals: work for anything**

Population Prediction Intervals

Approach:

- Randomly draw from fitted parameter combinations
- Evaluate the function at each combination
- Calculate the CI of those values

Population Prediction Intervals

```
library(MASS)

# Draw from the parameter distribution

vmat <- mvrnorm(1000,

                mu = coef(w1),

                Sigma = vcov(w1))
```

Run this and inspect the vmat object

Population Prediction Intervals

Prepare an output object

```
survival_time_estimates <- numeric(nrow(vmat))
```

Write a function to calculate the mean

```
meanfun <- function(shape, scale) {  
  scale * gamma(1 + 1/shape)  
}
```

Population Prediction Intervals

For loop to calculate the mean at each value:

```
for(i in 1:length(survival_time_estimates)){  
  survival_time_estimates[i] <- meanfun(vmat[i, 1], vmat[i, 2])  
}
```

Calculate the CI using quantiles

```
survival_CI <- quantile(survival_time_estimates,  
                        c(0.025, 0.975))
```

Run this. What are the CI limits you found?

Population Prediction Intervals

You now have an estimate for survival, as well as a 95% CI

- What if we have a more complicated function?

Clis on a linear model

We're going to grab a different dataset for a more complicated model:

```
library(readr)
library(tidyverse)

avo <- read_rds("https://tinyurl.com/avonetdata") %>%
  filter(Family1 == "Momotidae") %>%
  arrange(avo$Beak.Length_Nares)
```

This code gets the data for Motmots and arranges it by Beak length

CIs on a linear model

Now, we fit a model:

```
avo_model <- mle2(Beak.Length_Culmen ~  
                  dnorm(mean = int + Beak.Length_Nares*b,  
                        sd = sd),  
                  start = list(int=1, b=1, sd=2),  
                  data=avo)
```

Run this and make sure it fit alright

CIs on a linear model

As before, we'll sample from 1000 draws from the model coefficient surface

```
avo_vmat <- mvrnorm(1000,  
                    mu = coef(avo_model),  
                    Sigma = vcov(avo_model))
```

We need an output **matrix** since we'll have multiple estimates (1 per species)

```
avo_estimates <- matrix(nrow = 1000,  
                        ncol = nrow(avo))
```

CIs on a linear model

As before, we'll sample from 1000 draws from the model coefficient surface

```
avo_vmat <- mvrnorm(1000,  
                    mu = coef(avo_model),  
                    Sigma = vcov(avo_model))
```

We need an output **matrix** since we'll have multiple estimates (1 per species)

```
avo_estimates <- matrix(nrow = 1000,  
                        ncol = nrow(avo))
```

CIs on a linear model

As before, we use a `for()` loop to evaluate the function at each value

- However, this time we get multiple values at each step (one per species)

```
for(i in 1:nrow(avo_estimates)){  
  parms_i <- avo_vmat[i,]  
  estimates_i <- parms_i[1] + parms_i[2]*avo$Beak.Length_Nares  
  avo_estimates[i,] <- estimates_i  
}
```

Run this and inspect the output

CIs on a linear model

We now have 1000 estimates per each species.

We can summarize this with a for() loop (although apply would be faster)

```
avo_estimates_ci <- NULL

for(i in 1:ncol(avo_estimates)){

  out_i <- data.frame(mean = mean(avo_estimates[,i]),
                      low = quantile(avo_estimates[,i], 0.025),
                      high = quantile(avo_estimates[,i], 0.975))

  avo_estimates_ci <- rbind(avo_estimates_ci,out_i)

}
```

Plotting our function's CI

```
plot(x = avo$Beak.Length_Nares,  
     y = avo$Beak.Length_Culmen)
```

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_estimates_ci$mean,  
      col = "blue")
```

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_estimates_ci$low,  
      col = "blue", lty = 2)
```

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_estimates_ci$high,  
      col = "blue", lty = 2)
```

Try this out, how does our fit do?

CIs: the easy way

We can use the “predict” function to get confidence intervals!

```
avo_ci <- predict(avo_lm,  
                  interval = "confidence",  
                  level = 0.95)
```

Try this and check out the output

Cls: the easy way

Let's visualize this:

```
plot(x = avo$Beak.Length_Nares,  
     y = avo$Beak.Length_Culmen)
```

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_ci[,1], col = "red")
```

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_ci[,2],  
      col = "red", lty = 2)
```

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_ci[,3],  
      col = "red", lty = 2)
```


CIs: the easy way

We can add our previous fits on top of this:

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_estimates_ci$mean, col = "blue")
```

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_estimates_ci$low,  
      col = "blue", lty = 2)
```

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_estimates_ci$high,  
      col = "blue", lty = 2)
```

How similar are the two estimates?

Prediction Intervals

Confidence intervals and population prediction intervals

- Focus on fitted model
- Ignore residual variance

If we want to predict future data, we often want to account for residual variance

- We can use prediction intervals for that
- Can use the “predict” function again

Prediction Intervals

Fit and plot out prediction interval

```
avo_pi <- predict(avo_lm,  
                  interval = "prediction",  
                  level = 0.95)
```

```
plot(x = avo$Beak.Length_Nares,  
     y=avo$Beak.Length_Culmen)  
lines(x = avo$Beak.Length_Nares,  
      y = avo_pi[,1],col="purple")  
lines(x = avo$Beak.Length_Nares,  
      y =avo_pi[,2],  
      col="purple",lty=2)  
lines(x = avo$Beak.Length_Nares,  
      y =avo_pi[,3],  
      col="purple",lty=2)
```

Prediction Intervals

Add our CI lines to the plot to compare

```
lines(x = avo$Beak.Length_Nares,  
      y = avo_ci[,1],col="red")
```

```
lines(x = avo$Beak.Length_Nares,  
      y =avo_ci[,2],  
      col="red",lty=2)
```

```
lines(x = avo$Beak.Length_Nares,  
      y =avo_ci[,3],  
      col="red",lty=2)
```

Take-aways

- CIs can display uncertainty in your fitted model
 - Predict is easier
 - PPIs can be useful when predict doesn't work
- PIs display uncertainty in future data (model + residuals)
 - Always wider than CIs and PPIs
 - Useful when making predictions or for simulations

Before next class:

- Skim chapter 8